Monte Carlo Simulation - Portfolio loss and importance sampling

Thomas Sejer Canham

October 17, 2026

Aarhus University

Contents

1 Introduction **1**

2 Crude Monte Carlo Simulation (a) **1**

3 Control Variable (b) **2**

4 Importance-Sampling (c) **5**

5 Comparison of the methods (d) **7**

6 Closed form expressison of $\mathbb{P}(L > x)$ for the given parameters (e) **9**
6.1 Is this true for other distributions? 9
6.2 Generalization of crude and control- MCS to other distributions 10

7 Cholesky Decomposition (f) **10**
7.1 Eigen decomposition of $\tilde{C}^T\tilde{C}$ 10
7.2 New expression of L 11

8 Repetition of (a) and (b) with new parameters **11**
8.1 Repetition of (a) 12
8.2 Repetition of (b) 12

9 Exponential tilting (g) **12**

10 Appendix **13**

1 Introduction

In this project, we will investigate the use of Monte Carlo simulation (MCS) to estimate the exceedance probability $\mathbb{P}(L > x)$ of a loss

$$L = \delta^T \Delta S + \Delta S^T \Gamma \Delta S,$$

where, $delta \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim \mathcal{N}(0, \Sigma_S)$ is a (multivariate) normal random variable. The exceedance probability is the probability that the loss L exceeds a certain threshold $x = 10$.

In the parts (a), (b) and (c), we will focus on the one-dimensional case, $m = 1$. These three parts will compare crude MCS with MCS where we introduce control variates and importance samplingm respectively. In part (d), the differences of these methods are visuallized by comparison of the confidence intervals of the estimates that are obtained from each method.

Parameters for the various codes snippets can be seen in the appendix in Listing 6.

2 Crude Monte Carlo Simulation (a)

In this section, we will implement a crude Monte Carlo simulation to estimate the exceedance probability $\mathbb{P}(L > x)$ for the one-dimensional case ($m = 1$). We are interested in $P(L > x)$. We may introduce the indicator function $\mathbb{I}(L > x)$ of the event $L > x$. If we compute the expectation of the indicator function, we obtain the exceedance probability

$$\mathbb{P}(L > x) = \mathbb{E}[\mathbb{I}(L > x)] = 0 \cdot \mathbb{P}(L > x) + 1 \cdot \mathbb{P}(L > x) = \mathbb{P}(L > x), \tag{1}$$

by the bernoulli property of the indicator function. We estimate the exceedance probability by use of the following Monte Carlo estimator

$$\hat{\ell} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(L_i > x) \xrightarrow[N \to \infty]{a.s} \mathbb{E}[\mathbb{I}(L > x)], \tag{2}$$

by the law of large numbers. The crude Monte Carlo simulation is implemented in the code snippet seen in Listing 1. A estimate of

$$\hat{\ell} \approx 0.0103$$

is found with a variance of $s^2 = 1.03 \cdot 10^{-7}$.

Listing 1: Python code for crude MCS

```
"""
Code for task (a) - Crude monte carlo

"""
from parameters import *
np.random.seed(0)
N = 100000
if m == 1:
    sampling_dist = np.random.normal(loc = 0, scale = 1, size = (1,N))
if m == 2:
    sampling_dist = np.random.multivariate_normal(np.zeros(2), sigma_s, size = N
        ).T


def crude_monte_carlo(N, sampling_dist):
    """
    Crude Monte Carlo simulation to estimate the probability of exceedance of x.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """
```

```
24
25
26      # Simulate N outcomes of L
27      loss = L(sampling_dist)
28
29      # Mean of indicator converges a.s to P(L > x)
30      exeedance = indicator(x, loss)
31      # Estimate
32      exeedance_prop = np.mean(exeedance)
33      # Variance of estimate
34      variance = np.var(exeedance) / N
35      return exeedance_prop, variance
36
37  if __name__ == "__main__":
38      print("Crude Monte Carlo simulation: \n")
39      # Estimate and variance of estimator
40      l, s2 = crude_monte_carlo(N, sampling_dist)
41      print("Estimated probability of exceedance of x: ", l)
42      print("Variance of MC estimator: ", s2)
```

3 Control Variable (b)

A control variable is now introduced with the aim of reducing the variance of the Monte Carlo estimator. The estimator is now given by

$$\ell_{CV} = Y - \beta(Z - \mathbb{E}[Z]), \tag{3}$$

where $Y = \mathbb{I}(L > x)$ is the original estimator, Z is the control variable and β is a constant. It follows from proposition 5.1, that the optimal value of β is given by

$$\beta^* = \frac{\mathrm{Cov}(Y, Z)}{\mathrm{Var}(Z)}.$$

When considering the choice of a control variable, we should aim to choose a variable that satisfies the following properties:

1. Closed form expression for the expectation of the control variable $\mathbb{E}[Z]$.
2. The control variable should be as correlated with the original estimator Y as possible.
3. The reduction in variance should be significant enough to justify the additional computational cost of calculating the control variable.

Given L, two choices of control variables are natural. The first one is the linear term of L given by:

$$Z_1 = \delta^T \Delta S,$$

and the second one is the quadratic term of L given by:

$$Z_2 = \Delta S^T \Gamma \Delta S.$$

Since m, $\Gamma = 1$, and the fact that $Delta_S$ is standard normal, $Z_1 \sim \delta\mathcal{N}(0, 1)$ and $Z_2 \sim \chi^2(1)$, and as such their expectations are given by

$$\mathbb{E}[Z_1] = 0, \quad \mathbb{E}[Z_2] = 1.$$

To test which control variable is the best, a small pilot study is conducted to find the sample correlation between the original estimator Y and the two control variables Z_1 and Z_2, yeilding

$$\mathrm{Corr}(Y, Z_1) \approx 0.25, \quad \mathrm{Corr}(Y, Z_2) \approx 0.37.$$

Given the results of the pilot study, and the fact that Z_2 is not much more computationally expensive to compute than Z_1, Z_2 is chosen as a control variable. This leads to a reduction in variance of approximately 26% compared to the crude MCS. Since crude MCS has a $\sqrt{N}$ convergence rate, and we we roughly double the number of operations by computing the control variable, it seem reasonable to stick with crude MCS and double the number of samples, in order to achieve a reduction in variance of the order $\sqrt{2}$.

Listing 2: Python code for control variable MCS

```
"""
We introduce a control variable to reduce the varinace.
"""
# Same as in (a)
from a import crude_monte_carlo
from parameters import *


N = 10000
if m == 1:
    sampling_dist = np.random.normal(loc = 0, scale = 1, size = N)
if m == 2:
    sampling_dist = np.random.multivariate_normal(np.zeros(2), sigma_s, size = N
        )

# Use Q as control variable
def Q(Delta_s,  delta):
    return delta.T @ Delta_s

def compute_optimal_coefficients(Y, Z):
    """
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        beta_opt: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")

    cov = np.cov(np.vstack([Y, Z]), ddof=1)

    cov_YZ = cov[0, 1:]
    cov_ZZ = cov[1:, 1:]

    beta_opt = np.linalg.solve(cov_ZZ, cov_YZ)
    return beta_opt

def compute_correlation(Y, Z):
    """
    Compute the sample correlation coefficient for k different control variates.
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        correlation: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")

    cov = np.cov(np.vstack([Y, Z]), ddof=1)

    cov_YZ = cov[0, 1:]
    var_Y = cov[0, 0]
    var_Z = np.diag(cov[1:, 1:])
```

```
    correlation = cov_YZ / np.sqrt(var_Y * var_Z)
    return correlation




def control_monte_carlo(N, sampling_dist):
    """
    Control variate Monte Carlo simulation to estimate the probability of
        exceedance of x.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """

    # Simulate N outcomes of L
    loss = L(sampling_dist)

    # Compute control variable

    #Linear term, corr aprrox 0.25
    # #Z = Q(Delta_s, delta)

    # Quadratic term, corr approx 0.4
    Z = L(sampling_dist) - Q(sampling_dist, delta)
    # Expectation of control variable
    #E_Z = 0 # Linear term is N(0,1) * delta -> E[Z] = 0
    if m == 1:
        E_Z = 1 # Quadratic term i Chi_squared(1) -> E[Z] = 1
    if m == 2:
        lam1 = 1/2
        lam2 = 3/2
        # (f) gives weighting of eigenvalues of two chi_squared(1)
        E_Z = lam1 * 1 + lam2 * 1


    # Mean of indicator converges a.s to P(L > x)
    exeedence = indicator(x, loss)


    # optimal coeffcient
    beta_opt = compute_optimal_coefficients(exeedence, Z)

    # Control estimator
    exeedence_tilde = exeedence -  beta_opt*(Z - E_Z)

    # Estimate
    exeedance_prop = np.mean(exeedence_tilde)

    # Variance of estimate
    variance = np.var(exeedence_tilde) / N

    return exeedance_prop, variance

if __name__ == '__main__':
    # Correlation subrun
    np.random.seed(0)
    M = 1000
    Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=M).T
    loss = L(Delta_s, delta, gamma)
```

```
    Y = indicator(x, loss)
    # Compute correlation between linear approximation Q, and second term of L
    Z = np.zeros((2,M))
    Z[0,:] = Q(Delta_s, delta)
    Z[1,:] = loss - Q(Delta_s, delta)
    correlation = compute_correlation(Y, Z)
    print("Sample Correlation of linear term and exceedance indicator: ",
        correlation[0])
    print("Sample Correlation of quadratic term and exceedance indicator: ",
        correlation[1])

    print("Control variate Monte Carlo simulation: \n")
    N = 10000
    Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=N).T
    # Estimate and variance of estimator
    l_crude, s2_crude = crude_monte_carlo(N, Delta_s)
    l_control, s2_control = control_monte_carlo(N, Delta_s)
    print("Estimated probability of exceedance of x using control variable: ",
        l_control)
    print("Variance of MC estimator using control: ", s2_control)
    print()
    print("Variance reduction factor: ", s2_crude / s2_control)
```

4 Importance-Sampling (c)

Since we are interested in estimating a relatively rare event, we can use importance sampling to reduce the variance. We are interested in designing a new distribution under which the event $L > x$ is likely to occur. For numerical stability, it is desirable to choose a distribution that is not too far from the original distribution. If we inspect L, it is compromised of a linear and a quadratic term. We may consider if we want to shift the mean of Δ_S upwards or downwards. Since we have a quadratic term, that is always positive, shifting the mean in a negative direction, will be somewhat dampened by the linear term, as such we would require larger shifting in the negative direction to observe the same effect, therefore, a positive shift is chosen. The expexctation of the loss L is given by

$$\mathbb{E}[L] = \mathbb{E}[\delta^T \Delta S + \Delta S^T \Gamma \Delta S] = \mathbb{E}[\delta^T \Delta S] + \mathbb{E}[\Delta S^T \Gamma \Delta S] = 0 + 1 = 1,$$

Now we would like to shift the mean of ΔS such that the expectation of L is equal to the threshold $x = 10$, such that the rare event $L > x$, is no longer rare. We do this by setting the expectation of L under the new distribution $\mathbb{Q}$ to be equal to $x = 10$:

$$\mathbb{E}_{\mathbb{Q}}[L] = \mathbb{E}[\delta^T(\Delta S + \mu) + (\Delta S + \mu)^T \Gamma(\Delta S + \mu)] = 10.$$

as adding a mean μ to ΔS, will shift its mean by μ, so for $m = 1$

$$\mathbb{E}_{\mathbb{Q}}[L] = 2\mu + \mathbb{E}(\Delta S^2) + \mu^2 = 2\mu + 1 + \mu^2 = 10 \implies \mu(\mu + 2) = 9.$$

The positive solution to this equation is $\sqrt{(10)} - 1 \approx 2.16$, and in accordance with above reasoning, we see that a negative shift would require $\mu \approx -4.16$, which is considerably larger in magnitude.

Using this design choice, variance reduction on the order of 3000% is achieved. As such, importance-sampling offers a significant improvement compared to crude MCS, far outweighing the additional computational cost of computing the likelihood ratios. The code snippet for the importance sampling MCS can be seen in Listing 3.

Listing 3: Python code for importance sampling MCS

```
"""
Monte Carlo with importance sampling for variance reduction
"""
from a import indicator, L
from parameters import *
```

```
N = 10000
np.random.seed(0)
sampling_dist = np.random.normal(loc = 0, scale = 1, size = (1, N))

def likilihood_ratio(X, mu_p, sigma_p, mu_q, sigma_q):
    """
    We use another normal distribution with greater mean as importance sampling.
    Compute dP(X) / dQ(X), where:
    Under P: X_i ~ N(m_P, variance)
    Under Q: X_i ~ N(m_P + drift, variance)
    """
    # Compute the likelihood ratio
    p = (1 / np.sqrt(2 * np.pi * sigma_p**2)) * np.exp(-0.5 * ((X - mu_p) / sigma_p)**2)
    q = (1 / np.sqrt(2 * np.pi * sigma_q**2)) * np.exp(-0.5 * ((X - mu_q) / sigma_q)**2)
    return p / q


import numpy as np

def importance_monte_carlo(
    N,
    importance_sample,
    likelihood_ratio_func,
    likelihood_ratio_args=None,
    use_sigma_s = True,
):
    """
    Estimate probability of exceedance using importance sampling.

    Parameters
    ----------
    N : int
        Number of simulations.
    likelihood_ratio_func : callable
        Function that computes likelihood ratios.
    likelihood_ratio_args : dict, optional
        Keyword arguments passed to likelihood_ratio_func.
    importance_sample : ndarray, optional
        Samples drawn from the importance distribution.

    Returns
    estimate of exceedance probability.
    variance of estimator.
    """

    if likelihood_ratio_args is None:
        likelihood_ratio_args = {}

    # Compute likelihood ratios
    weights = likelihood_ratio_func(**likelihood_ratio_args)
    print("Mean of weights: ", np.mean(weights))

    # Simulate N outcomes of L
    loss = L(importance_sample, use_sigma_s = use_sigma_s)

    # Mean of indicator converges a.s to P(L > x), weighted by the likihood ratio
    exeedance = indicator(x, loss) * weights
    # Estimate
    exeedance_prop = np.mean(exeedance)
    # Variance of estimate
    variance = np.var(exeedance) / N
```

```
    return exeedance_prop, variance

if __name__ == '__main__':

    from a import crude_monte_carlo
    l_crude, s2_crude = crude_monte_carlo(N, sampling_dist)
    print("Importance sampling Monte Carlo simulation: \n")
    importance_sample = np.random.multivariate_normal(
        mu_q, sigma_s, size=N
    ).T
    lr_args = {"X" : importance_sample,
               "mu_p" : mu_p,
               "sigma_p" : sigma_p,
               "mu_q" : mu_q,
               "sigma_q" : sigma_q,}

    l_importance, s2_importance = importance_monte_carlo(N,
                                                         importance_sample,
                                                         likelihood_ratio_func =
                                                               likilihood_ratio,
                                                         likelihood_ratio_args=
                                                            lr_args)

    print("Estimated probability of exceedance of x using importance sampling: "
        , l_importance)
    print("Variance of MC estimator using importance sampling: ", s2_importance)
    # Variance reduction factor
    print("The variance reduction factor of importance sampling: ", s2_crude /
        s2_importance)
```

5 Comparison of the methods (d)

To compare the three methods, we examine the confidence intervals for 30 different simulations runs with distinct random seeds. The results are shown in Figure 1.

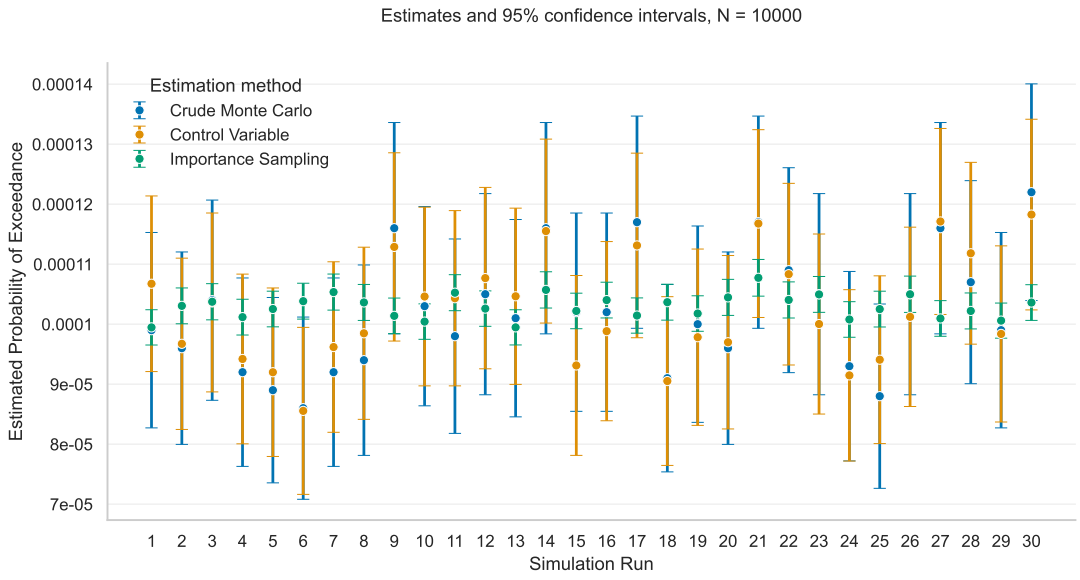


Figure 1: Comparison of confidence intervals for crude MCS, control variable MCS and importance sampling MCS. The confidence intervals are based on 30 simulations with $N = 10^4$ samples each. Dots represent the estimated probabilities.

We see that the confidence intervals are much narrower for the importance sampling MCS, which is consistent with the previous results.

The code to produce the plot can be seen in Listing 4.

Listing 4: Python code for comparison of the three methods

```python
import numpy as np
import matplotlib.pyplot as plt
from a import crude_monte_carlo
from b import control_monte_carlo
from c import importance_monte_carlo
from parameters import *
np.random.seed(0)

# Column is method, row is simulation run
estimates = np.zeros((SIMULATION_RUNS, len(methods)))
ci_lower = np.zeros_like(estimates)
ci_upper = np.zeros_like(estimates)

# Define seeds for reproducibility
random_seeds = range(0, SIMULATION_RUNS*len(methods))

for j, m in enumerate(methods):
    for i in range(SIMULATION_RUNS):
        np.random.seed(random_seeds[i])
        """
        Simulate the three methods and store the estimates and confidence
            intervals.
        """
        if m == "Crude Monte Carlo":
            # Simulate
            l, s2 = crude_monte_carlo(Nd)
        if m == "Control Variable":
            l, s2 = control_monte_carlo(Nd)
        if m == "Importance Sampling":
            l, s2 = importance_monte_carlo(Nd)

        s = np.sqrt(s2)
        # Compute confidence interval
        ci = confidence_interval(s, l, Nd)
        ci_lower[i, j] = ci[0]
        ci_upper[i, j] = ci[1]
        estimates[i, j] = l

# Mean interval lengths

for j,m in enumerate(methods):
    ci_lengths = ci_upper[:,j] - ci_lower[:, j]
    mean_ci_length = np.mean(ci_lengths)
    print("The mean CI length for method " + m + " is: ", mean_ci_length)

sns.set_theme(style="whitegrid", context="notebook")
fig, ax = plt.subplots(figsize=(10, 5.5))

# Horizontal offsets to separate the three methods
offsets = [0,0,0]

for j, method in enumerate(methods):
    x = plot_n + offsets[j]

    # Asymmetric error lengths: estimate - lower, upper - estimate
    yerr = np.vstack([
        estimates[:, j] - ci_lower[:, j],
        ci_upper[:, j] - estimates[:, j],
    ])*100

    ax.errorbar(
```

```
            x,
            estimates[:, j],
            yerr=yerr,
            fmt="o",
            color=colors[j],
            label=method,
            capsize=4,
            elinewidth=1.8,
            markersize=6,
            markeredgecolor="white",
            markeredgewidth=0.5,
        )

ax.set_xticks(plot_n)
ax.set_xlabel("Simulation Run")
ax.set_ylabel("Estimated Probability of Exceedance")
ax.set_title(f"Estimates and 95% confidence intervals, N = {Nd}\n", pad=14)

ax.legend(title="Estimation method", frameon=False)
ax.grid(axis="y", alpha=0.3)
ax.grid(axis="x", visible=False)
from matplotlib.ticker import FuncFormatter
ax.yaxis.set_major_formatter(
    FuncFormatter(lambda value, _: f"{value / 100:g}")
)


sns.despine()
fig.tight_layout()
# Save as SVG
plt.savefig("figures/ConfInts.pdf", format="pdf", bbox_inches="tight")
plt.show()
```

6 Closed form expressison of $\mathbb{P}(L > x)$ for the given parameters (e)

We wish to show that the exceedance probability can be computed in a closed form expression involving the CDF $\phi(x)$ of the standard normal distribution. Given the parameters, $m = 1$, $\delta = 2$, $\Gamma = 1$, $\Sigma_S = 1$ and $x = 10$, we have $\Delta_S \sim \mathcal{N}(0, 1)$ and $L = 2\Delta S + \Delta S^2$. We can rewrite the exceedance probability as

$$\mathbb{P}(L > x) = \mathbb{P}(\Delta S^2 + 2\Delta S - 10 > 0) \tag{4}$$

The left hand side is a quadratic polynomial in ΔS, we note that it is convex, as such, this probability is equivalent to the probablity that ΔS is lesser than the smaller root of the polynomial or greater than the larger root of the polynomial. The roots of the polynomial are given by

$$r_- = -1 - \sqrt{11}, \quad r_+ = -1 + \sqrt{11}.$$

We note that these events are disjoint, so we can write the exceedance probability as

$$\mathbb{P}(L > x) = \mathbb{P}(\Delta S < r_- \cup \Delta S > r_+) = \mathbb{P}(\Delta S < r_-) + \mathbb{P}(\Delta S > r_+) \tag{5}$$

Since $\Delta S \sim \mathcal{N}(0, 1)$, we can express the exceedance probability in terms of the CDF $\phi(x)$ of the standard normal distribution as

$$\mathbb{P}(L > x) = \phi(r_-) + 1 - \phi(r_+) \approx 0.0103, \tag{6}$$

which is consistent with the results obtained from the Monte Carlo simulations.

6.1 Is this true for other distributions?

We imposed or used no special restrictions on the distribution of ΔS, as such, this method could be used to compute the exceedance probability under any distribution, as long as the CDF of the distribution is available.

6.2 Generalization of crude and control- MCS to other distributions

As long as we can sample from the distribution of ΔS, we can use crude MCS to estimate the exceedance probability without any modifications.

For control variable MCS, we do have the requirement that the expectation of the control variable is known in closed form - however this is true in general, and changing the sampling distribution of ΔS does not change this fact. As such, the control variable MCS can be generalized to other distributions without any modifications other than that the control variable $Z = h(\Delta S)$ has to be chosen such that its expectation $\mathbb{E}[Z]$ is known in closed form.

7 Cholesky Decomposition (f)

We wish to find a matrix $\tilde{C}$ such that $\Delta_S = \tilde{C}Z$, where $Z \sim \mathcal{N}(0, I_2)$, it follows from definition 3.8 that we can find such a matrix $\tilde{C}$ by performing a Cholesky decomposition of the covariance matrix Σ_S. We define the upper triangular matrix $\tilde{C}$ as

$$\tilde{C} = \begin{bmatrix} a & 0 \\ b & c \end{bmatrix},$$

with unknowns $a, b, c \in \mathbb{R}$, and solve

$$\tilde{C}\tilde{C}^T = \begin{bmatrix} a & 0 \\ b & c \end{bmatrix} \begin{bmatrix} a & b \\ 0 & c \end{bmatrix} = \Sigma_S = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{bmatrix}. \tag{7}$$

This produces the following system of equations

$$\begin{aligned} a^2 &= 1 \\ ab &= \frac{1}{2} \\ b^2 + c^2 &= 1. \end{aligned}$$

$a^2 = 1 \implies a = 1$, and thus $b = \frac{1}{2}$. We can than solve $\frac{1}{4} - c^2 = 1 \implies c = \sqrt{1 - \frac{1}{4}} = \sqrt{\frac{3}{4}}$.

$$\tilde{C} = \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & \sqrt{\frac{3}{4}} \end{bmatrix}.$$

7.1 Eigen decomposition of $\tilde{C}^T\tilde{C}$

First we compute $\tilde{C}^T\tilde{C}$:

$$\tilde{C}^T\tilde{C} = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & \sqrt{\frac{3}{4}} \end{bmatrix} \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & \sqrt{\frac{3}{4}} \end{bmatrix} = \begin{bmatrix} \frac{5}{4} & \frac{1}{2}\sqrt{\frac{3}{4}} \\ \frac{1}{2}\sqrt{\frac{3}{4}} & \frac{3}{4} \end{bmatrix}. \tag{8}$$

We note that $\tilde{C}^T\tilde{C}$ is a real symmetric matrix, it follows from the spectral theorem that there exists and orthogonal matrix U and a diagonal matrix Λ such that

$$\tilde{C}^T\tilde{C} = U\Lambda U^T. \tag{9}$$

where the columns of U are the eigenvectors of $\tilde{C}^T\tilde{C}$ and the diagonal entries of Λ are the corresponding eigenvalues. Thus, we need to solve the eigenvalue problem

$$\det(\tilde{C}^T\tilde{C} - \lambda I) = 0, \tag{10}$$

which yields the characteristic polynomial

$$\left(\frac{5}{4} - \lambda\right)\left(\frac{3}{4} - \lambda\right) - \left(\frac{1}{2}\sqrt{\frac{3}{4}}\right)^2 = 0.$$

This yeilds the eigenvalues $\lambda_1 = \frac{1}{2}$, $\lambda_2 = \frac{3}{2}$, now we find the eigenvectors by solving the system of equations

$$(\tilde{C}^T\tilde{C} - \lambda I)u = 0,$$

which yields the eigenvectors

$$u_1 = \begin{bmatrix} \sqrt{\frac{3}{4}} \\ \frac{1}{2} \end{bmatrix}, \quad u_2 = \begin{bmatrix} -\frac{1}{2} \\ \sqrt{\frac{3}{4}} \end{bmatrix}.$$

and U is given as the matrix with columns u_1 and u_2, and Λ is the diagonal matrix with entries λ_1 and λ_2.
The computation is carried out in the code snippet seen in Listing 5.

Listing 5: Python code for eigen decomposition of $\tilde{C}^T\tilde{C}$

```python
# Python code for eigen decomposition of C^T C
import numpy as np

# Define the matrix
C_T_C = np.array([[5/4, 0.5*np.sqrt(3/4)],
                  [1/2*np.sqrt(3/4), 3/4]])

# Compute the eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(C_T_C)

print("Eigenvalues:", eigenvalues)
print("Eigenvectors:", eigenvectors)
```

7.2 New expression of L

Using our Cholesky decomposition, we have $\Delta S = \tilde{C}Z$, where $Z \sim \mathcal{N}(0, I_2)$. It then follows that we can rewrite L as

$$L = \delta^T\tilde{C}Z + Z^T\tilde{C}^T\Gamma\tilde{C}Z. \tag{11}$$

Now since $\Gamma = I_2$, this simplifies to

$$L = \delta^T\tilde{C}Z + Z^T\tilde{C}^T\tilde{C}Z. \tag{12}$$

We define $Z = [Z_1, Z_2]^T$, and use the eigen decomposition (9) to rewrite L as

$$L = \delta^T\tilde{C}Z + Z^TU\Lambda U^TZ = \delta^T\tilde{C}Z + (U^TZ)^T\Lambda U^TZ. \tag{13}$$

Not let $C = \tilde{C}U \implies C^T = U^T\tilde{C}^T$, and $b = C^T\delta = U^T\tilde{C}^T\delta$. If we look at the first term of L, and use the fact that U is orthogonal, we have

$$\delta^T\tilde{C}Z = (UU^T\tilde{C}^T\delta)^TZ = (Ub)^TZ = b^TU^TZ. \tag{14}$$

Now it is useful to note that the linear transformation $U^TZ \sim \mathcal{N}(0, U^TU)$, orthogonality then gives that $U^TZ \sim \mathcal{N}(0, I_2) = Z$. As such (14) simplifies to

$$\delta^T\tilde{C}Z = b^TU^TZ = b^TZ. \tag{15}$$

Using this property, again we can rewrite (13) as

$$L = b^TZ + (U^TZ)^T\Lambda U^TZ = b^TZ + Z^T\Lambda Z, \tag{16}$$

using (14). The first term is a inner product of two vectors, and the since Λ is diagonal, the second term is a weighted sum of squares of the entries of Z - as such we arrive at the desired expression of L:

$$L = b_1Z_1 + b_2Z_2 + \lambda_1Z_1^2 + \lambda_2Z_2^2. \tag{17}$$

8 Repetition of (a) and (b) with new parameters

(a) and (b) are repeated with the new parameters, $m = 2$, $\delta = [2, 2]^T$, $\Gamma = I_2$, $\Sigma_S = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{bmatrix}$ and $x = 15$.
Code wise the implementation is identical, however the m variable in the *parameters.py* file is changed to $m = 2$, this updates δ, Γ and Σ_S accordingly.

8.1 Repetition of (a)

For $N = 10^5$, we obtain an estimate of $\hat{\ell} \approx 0.0153$ with a variance of $s^2 \approx 1.5134 \cdot 10^{-7}$.

8.2 Repetition of (b)

For $N = 10^5$, we obtain an estimate of $\hat{\ell} \approx 0.0153$ with a variance of $s^2 \approx 1.1693 \cdot 10^{-7}$. One change in (b) is that we have to update the expectation of the control variable $Z_{control} = \Delta S^T \Gamma \Delta S$. However, it follows from (17) that the expectation of Z is given by

$$\mathbb{E}[Z_{control}] = \mathbb{E}[\lambda_1 Z_1^2 + \lambda_2 Z_2^2] = \lambda_1 \mathbb{E}[Z_1^2] + \lambda_2 \mathbb{E}[Z_2^2] = \lambda_1 \mathbb{E}[\chi_1^2] + \lambda_2 \mathbb{E}[\chi_1^2] = \lambda_1 + \lambda_2 = 2,$$

where $Z_1, Z_2 \sim \mathcal{N}(0,1)$ and $\lambda_1 = \frac{1}{2}, \lambda_2 = \frac{3}{2}$ are the eigenvalues of $\tilde{C}^T \tilde{C}$, from (f). This is handled automatically if m is set to 2 in the *parameters.py* file. This time we see a variance reduction of $\approx 23.0\%$.

9 Exponential tilting (g)

We are now interested in importance sampling similar to (c), but for the new parameters. In the previous section we have shown that we can express L as a function of Z, rather that ΔS, this allows us to perform importance sampling from multivariate normal with identity covariance matrix. Recall that in extension to exercise 27, we discussed that exponential tilting for exponential families, retains distributional form under a new shifted parameter.

$$\int_A \prod_{i \leq 2} f(z) e^{\theta_i^T z - \psi^*(\theta_i)} dz, \tag{18}$$

Our original distribution is $Z \sim \mathcal{N}(0, I_2)$, and we can write the density of Z as

$$f(z) = \frac{1}{2\pi} e^{-\frac{1}{2} z^T(z)}.$$

We can then introduce the titlted distribution with parameter $\theta = (\theta_1, \theta_2) \in \mathbb{R}^2$ as

$$f(z) e^{\theta_i^T z - \psi^*(\theta_i)} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2} z^2} e^{\theta_i z - \frac{1}{2}\theta_i^2} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(z-\theta_i)^2}.$$

We see that the new importance sampling distribution from (18) is a joint normal distribution(Q_1, Q_2), where $Q_i \sim \mathcal{N}(\theta_i, 1)$, and $Q_1 \perp Q_2$. As such the likelihood ratio is given by

$$\frac{f(z)}{g(z)} = \frac{\prod_{i \leq 2} f(z_i)}{\prod_{i \leq 2} g(z_i)} = \prod_{i \leq 2} e^{-\theta_i z_i + \frac{1}{2}\theta_i^2}. \tag{19}$$

10 Appendix

Listing 6: parameters.py - Parameters for the various code snippets

```python
import numpy as np

### (a), (b) ###
# Set m = 1 for (a) and (b) and m = 2 for repetition of (a) and (b) with m = 2
m = 2

# delta, gamma is a column vector of size m
delta = np.ones((m, 1)) * 2
gamma = np.eye(m)

def indicator(x, L):
    # Exceedance of x
    return L > x




if m == 1:

    sigma_s = np.ones((m, m))

    x = 10

    def L(X, kw = None):
        """L for N simulations, keyword added for interchangeability of m = 1 or
            2
        input:
        N (int): Number of simulations
        X: of shape (m, N) samples
        gamma: vector of size m
        delta: vector of size m
        """
        # Returns (N,) losses

        L = delta.T @ X + np.sum(X * (gamma @ X), axis=0)
        return L


if m == 2:

    sigma_s = np.array([[1, 0.5], [0.5, 1]])



    x = 15

    # Define the matrix
    C_T_C = np.array([[0.75, 0.5 * np.sqrt(0.75)],
                      [0.5 * np.sqrt(0.75), 1.25]])

    # Compute the eigenvalues and eigenvectors
    eigenvalues, U = np.linalg.eig(C_T_C)
    C_tilde = np.array([[1, 1 / 2],
                        [0, np.sqrt(3 / 4)]])
    C = C_tilde @ U
    b = np.real(C.T @ delta)


    def L(X, use_sigma_s = True):
        """L for N simulations
```

```
        input:
        N (int): Number of simulations
        X: of shape (m, N) samples
        gamma: vector of size m
        delta: vector of size m
        """
        if use_sigma_s:
            L = delta.T @ X + np.sum(X * (gamma @ X), axis=0)
            return L
        else:
            # Loss computed in accordance with (f)
            lam1 = np.real(eigenvalues[0])
            lam2 = np.real(eigenvalues[1])
            L = b[0] * X[0, :] + b[1] * X[1, :] + lam1 * np.pow(X[0, :], 2) +
                lam2 * np.pow(X[1 , :], 2)
            return L

### (c) ###

# Q measure paramters
mu_q = np.ones(m) * np.sqrt(10) - 1
sigma_q = sigma_s

# P measure parameters
mu_p = np.zeros(m)
sigma_p = sigma_s

######## (d) ########
import seaborn as sns
from scipy.stats import norm

Nd = 10000

SIMULATION_RUNS = 30
plot_n = np.arange(1, SIMULATION_RUNS + 1)

methods = ["Crude Monte Carlo", "Control Variable", "Importance Sampling"]
colors = sns.color_palette("colorblind", 3)

def confidence_interval(s, l , N, level = 0.05):
    """Compute confidence interval, equation (1.2)"""
    z_alpha = norm.ppf(1 - level)
    return (l - z_alpha*s / np.sqrt(N), l + z_alpha*s / np.sqrt(N) )
```